

Daily Progress Report

Week 4_Day2: Promises & Fetch API Fundamentals

Program	3-Month MERN Stack Internship Program
Intern Name	Mohsin Ali
Week	Week_4 (Day 2)
Date	14 July, 2026

Day 2: Promises & Fetch API Fundamentals

1.1 Topics Studied

- JavaScript Promises and their lifecycle (Pending, Resolved/Fulfilled, and Rejected states)
- Handling asynchronous operations using `.then()` and `.catch()` methods
- Making HTTP GET requests using the native Fetch API
- Parsing API responses using `response.json()`
- Implementing error handling for failed network requests
- Dynamic DOM manipulation based on API responses
- Managing UI states (loading, success, empty, and error)
- Responsive layout design using CSS Grid
- Separation of concerns by organizing HTML, CSS, and JavaScript into modular files

1.2 Task Description

Developed a dynamic User Directory web application to practically implement JavaScript Promises and the native Fetch API. The application fetches mock user profile data from a public API (JSONPlaceholder) and dynamically renders it as a grid of profile cards on the DOM. The primary objective of this project was to move beyond basic scripts and build a robust, real-world implementation featuring distinct UI states (loading, success, error), proper error handling, and clean separation of concerns using HTML, CSS, and vanilla JavaScript.

1.3 Implementation Details

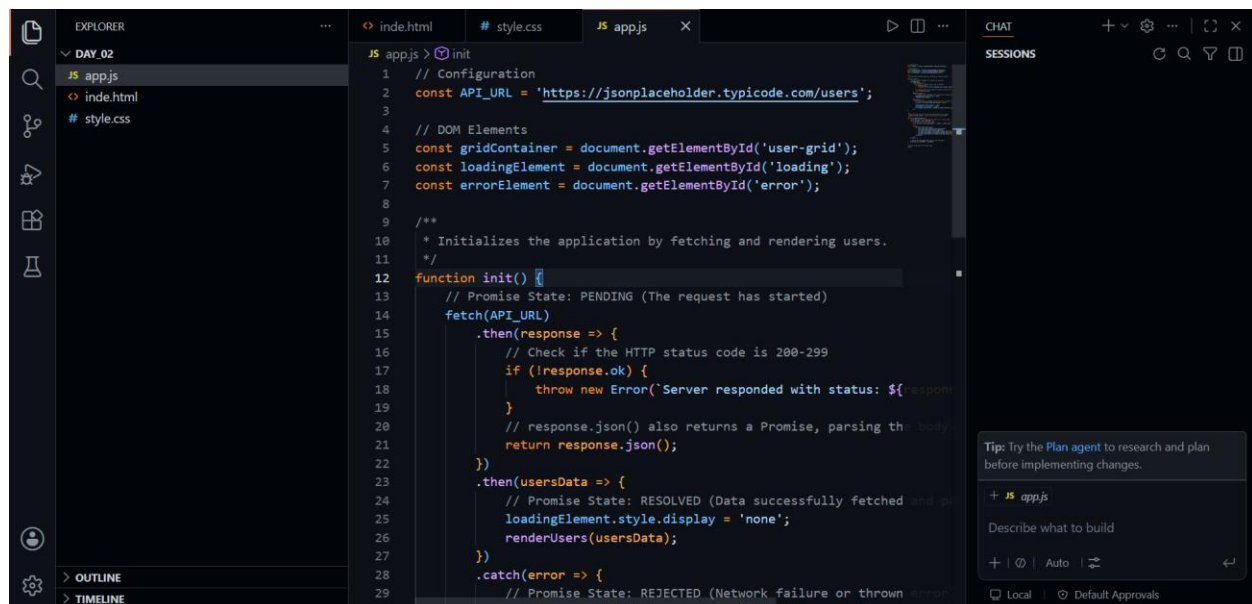
- Designed a clean, semantic HTML structure with dedicated containers for loading states, error messages, and the main data grid.

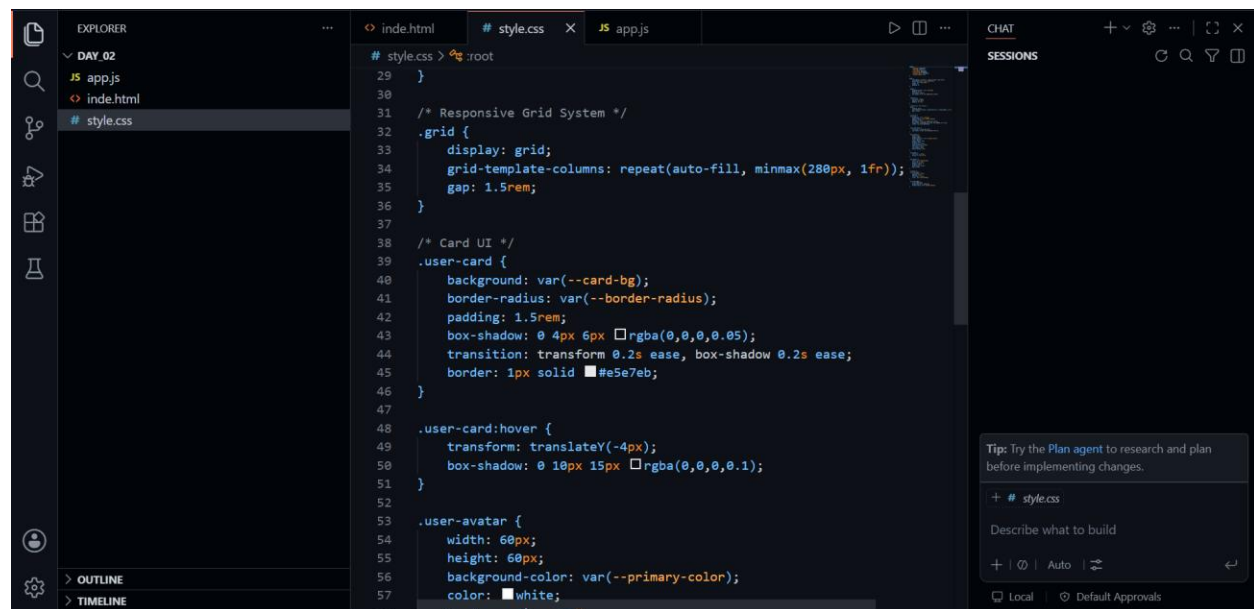
- Utilized the native `fetch()` API to execute HTTP GET requests to retrieve remote user data.
- Implemented `.then()` blocks to handle the successful resolution of Promises and efficiently parse the incoming JSON payload.
- Implemented a `.catch()` block to gracefully handle Promise rejections, ensuring network failures are caught and displayed as user-friendly messages rather than just console errors.
- Managed application UI states by dynamically displaying a loading indicator during the Pending state of the Promise, and hiding it once Resolved or Rejected.
- Iterated over the parsed JSON data using array mapping to generate dynamic HTML strings for individual user profile cards.
- Extracted and formatted user initials programmatically to create custom avatar placeholders for each profile.
- Updated the DOM efficiently by injecting the constructed HTML directly into the grid container without requiring page refreshes.
- Styled the application using modern CSS variables and a responsive CSS Grid system, ensuring the interface adapts seamlessly across desktop, tablet, and mobile devices.

1.4 Outcome

The User Directory application was successfully completed, demonstrating a practical and professional approach to asynchronous JavaScript. The project solidified my understanding of the Fetch API, Promise lifecycle management, and dynamic DOM rendering based on remote data. Furthermore, it reinforced front-end best practices by incorporating proper loading states and error handling, resulting in a robust, responsive, and user-friendly interface built with pure vanilla JavaScript.

Screenshots:





Screenshots:

